

Docker & Containers

Category: **Containerization & Runtime Packaging** | Platform Standard: **Linux 6+ / Modern Cloud SRE**

1. Domain Architectural Overview

This official module provides comprehensive engineering depth and runnable syntax for **Docker & Containers**. It is strictly scoped to this domain to prevent knowledge cross-contamination. Engineers mastering this domain acquire hands-on proficiency from foundational setup to production-grade incident triage and resilience.

2. Core Concepts & Production Principles

- **Linux Kernel Isolation Primitives (Namespaces & cgroups):** Containers are simply standard Linux processes isolated via namespaces (PID, NET, MNT, IPC, UTS, USER) and resource-metered via cgroups (CPU bandwidth, memory limits, blkio).

Production Application: Demystifies container troubleshooting: a container process can be inspected directly on the host using standard host tools (`ps`, `top`, `nsenter`, `/proc/`).

- **Multi-Stage Docker Builds & Image Layer Optimization:** Separating compilation environments (compilers, SDKs, build tools) from the final minimal runtime image containing only the compiled artifact and runtime dependencies.

Production Application: Reduces image size by up to 90% (e.g. from 1.2GB down to 35MB for Go/Node.js apps), slashing deployment pull times and minimizing attack surface.

- **Overlay2 Union Filesystem & Copy-on-Write (CoW):** Layering immutable image layers (`lowerdir`) with an ephemeral container read-write layer (`upperdir`) merged through a virtual mount (`merged`).

Production Application: Heavy disk writes inside a container without volumes degrade performance due to CoW overhead and can exhaust host root partition storage.

- **Container Lifecycle & Signal Handling (PID 1 Problem):** The root process inside a container runs as PID 1. If not properly configured (e.g. using `tini` or direct `exec`), PID 1 will not forward `SIGTERM` to child processes, causing slow 10-second timeouts on shutdown.

Production Application: Ensuring zero-downtime rolling updates by allowing container processes to gracefully finish active requests upon receiving `SIGTERM`.

3. Exhaustive Command & Syntax Reference

Command & Syntax	Purpose & Options	Production Usage & Common Pitfall
docker build -t payment-service:1.2.0 -f Dockerfile . <code>docker build [options]</code>	Build a tagged container image from a Dockerfile using the current directory as build context. Options: [{flag: '-t ', 'description': 'Assign a repository name and tag to the resulting image.'}, {flag: '-f ', 'description': 'Specify explicit path to Dockerfile.'}, {flag: '--no-cache', 'description': 'Force complete rebuild without using cached layers.'}, {flag: '--target ', 'description': 'Build up to a specific target stage in a multi-stage Dockerfile.'}]	Run in CI build stages with build caching enabled to maximize build velocity. Pitfall: Passing an excessively large build context (e.g. building from <code>`\$HOME`</code> or forgetting <code>`.dockerignore`</code>), resulting in gigabytes sent to Docker daemon.
docker run -d --name payment-api -p 8080:8080 --memory=512m --cpus=1.0 --restart=unless-stopped payment-service:1.2.0 <code>docker run [options] [command]</code>	Execute a container in detached background mode with bounded memory, CPU allocation, port mapping, and automatic restart policy. Options: [{flag: '-d', 'description': 'Run container in background and print container ID.'}, {flag: '--name', 'description': 'Assign a memorable name to the container.'}, {flag: '-p 8080:8080', 'description': 'Map host port 8080 to container port 8080.'}, {flag: '--memory=512m', 'description': 'Hard memory limit for the container cgroup.'}, {flag: '--cpus=1.0', 'description': 'Limit container CPU quota to 1.0 physical CPU core equivalent.'}, {flag: '--restart=unless-stopped', 'description': 'Restart container automatically on reboot or failure unless stopped manually.'}]	Always specify explicit resource limits (<code>--memory`</code>, <code>--cpus`</code>) to prevent a single rogue container from crashing the host. Pitfall: Running containers without memory limits, allowing memory leaks to trigger host-wide OOM killer panics.
docker logs --tail 100 -f payment-api <code>docker logs [options]</code>	Stream live container stdout and stderr logs starting from the last 100 lines. Options: [{flag: '-f', 'description': 'Follow log output continuously in real-time.'}, {flag: '--tail 100', 'description': 'Show only the most recent 100 lines of log output.'}, {flag: '--timestamps', 'description': 'Prepend ISO timestamps to each log line.'}]	Execute immediately upon container alert notification to inspect startup errors or exception stack traces. Pitfall: Running <code>`docker logs`</code> without <code>`--tail`</code> on long-running containers, flooding the terminal with millions of lines of historical logs.

Command & Syntax	Purpose & Options	Production Usage & Common Pitfall
docker exec -it payment-api /bin/sh <code>docker exec [options]</code>	Open an interactive terminal session inside a running container for live diagnostic inspection. Options: [{flag: '-i', 'description': 'Keep STDIN open for interactive typing.'}, {flag: '-t', 'description': 'Allocate a pseudo-TTY terminal.'}, {flag: '-u root', 'description': 'Optionally execute command as root user for privileged debugging.'}]	Use sparingly in staging/dev environments to test network reachability to database backends. Pitfall: Modifying application code or configuration permanently inside the container; all changes are lost when container restarts.
docker system prune -af --volumes <code>docker system prune [options]</code>	Clean up all stopped containers, unused networks, dangling and unreferenced images, and unattached volumes. Options: [{flag: '-a', 'description': 'Remove all unreferenced images, not just dangling ones.'}, {flag: '-f', 'description': 'Force execution without confirmation prompt.'}, {flag: '--volumes', 'description': 'Prune all unused anonymous and named volumes.'}]	Run on CI worker nodes where hundreds of ephemeral test containers and images accumulate daily. Pitfall: Running with `--volumes` in production environments where stateful database volumes are not actively mounted, permanently deleting data.

4. Security Hardening & Anti-Patterns

Security Mandates: Always operate under least-privilege principles. Restrict credentials, mask secrets in build outputs, avoid wildcard IAM permissions, and validate inputs before running commands.

Critical Anti-Pattern: Avoid manual drift ('click-ops' in cloud consoles or unversioned hotfixes in staging/production). Every change must be declared in code and reviewed via CI/CD.